DOCUMENTATION TECHNIQUE · V2.0.0

Simulation de voitures autonomes

Neuroévolution NEAT · Python + Flask · rendu navigateur

26	400	5
tests automatisés	frames de parité carte réelle	cartes disponibles

Ce document décrit l'architecture, les algorithmes et la démarche de refonte de la simulation de voitures autonomes. La physique et l'évolution NEAT du projet Pygame d'origine sont conservées à l'identique ; le rendu est déplacé dans le navigateur et le moteur devient pilotable en temps réel.

Points clés : architecture en modules, serveur Flask, flux temps réel SSE, corrections de robustesse, validation par tests et parité bit à bit.

Sommaire

1	Présentation et objectifs
2	Fonctionnalités
3	Architecture logicielle
4	Algorithmes de la simulation
5	Refactoring : avant / après
6	Corrections de bugs
7	Améliorations et performances
8	API HTTP et flux temps réel
9	Interface et prise en main
10	Validation par les tests
11	Limites et perspectives
A	Commandes utiles
B	Structure du projet

Les chiffres de ce document sont mesurés lors de sa génération (nombre de tests, taille des fixtures, coût d'une frame) : ils correspondent à l'état du code livré.

1. Présentation et objectifs

Le projet d'origine est une démonstration d'apprentissage évolutif : une population de voitures circule sur une piste vue du dessus, chacune pilotée par un réseau de neurones évolué par NEAT. Les meilleurs génomes se reproduisent, mutent et finissent par suivre la piste sans collision.

La demande était de corriger, refactoriser et améliorer l'application, en conservant Python et en passant à Flask pour le rendu dans un navigateur, sans modifier le comportement ni casser les algorithmes existants.

Ce qui est strictement conservé

- La physique : vitesse initiale de 20 px/frame, pas de rotation de 10°, accélération de 2, décélération plancher à 12.
- Les 5 capteurs (-90° à +90°, portée 300 px, entrée normalisée par 30).
- La détection de collision par les 4 coins et le pixel blanc.
- La récompense distance / 30 et son cumul par frame.
- Les paramètres NEAT de config.txt et la limite de 1 200 frames par génération.

Ce qui change

- Le rendu : plus de fenêtre Pygame, mais un canvas 1920 × 1080 dans le navigateur.
- Le pilotage : pause, reprise, vitesse et changement de carte à chaud.
- La robustesse : plus de crash sur les bords de carte, chemins absolus, état protégé par verrou.
- L'observabilité : API JSON, historique de fitness, génome sauvegardé.
- La qualité : 26 tests automatisés et une parité vérifiée avec l'original.

2. Fonctionnalités

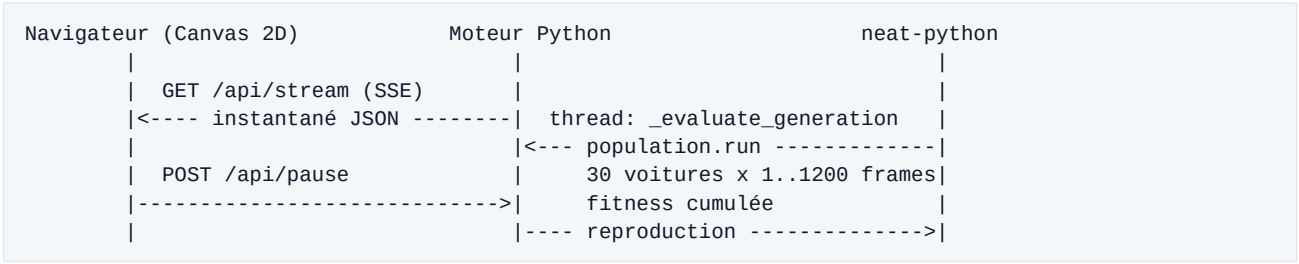
Fonction	Détail
Simulation temps réel	30 voitures par génération, 60 frames/s de référence, jusqu'à 1 000 générations
Contrôles	Démarrer, pause, reprendre, réinitialiser (boutons et raccourcis Espace / R)
Vitesse	0.5×, 1×, 2×, 4×, 8× — la physique ne change pas, seule la cadence
Cartes	5 pistes 1920 × 1080, changement à chaud avec redémarrage propre
Visualisation	Piste, sprite orienté, radars, textes de génération et de survie
Statistiques	Génération, vivantes, fitness courante, record, cadence, progression
Graphique	Courbe meilleure fitness / moyenne par génération
Génome	Meilleur génome sauvegardé dans runs/ et téléchargeable en .pkl
Temps réel	Flux SSE avec repli automatique en sondage HTTP
Entraînement	Mode headless train.py, même physique, sortie pickle

3. Architecture logicielle

Le script unique est remplacé par un paquet de six modules aux responsabilités nettes, un serveur Flask et un client Canvas. La physique reste en Python ; le navigateur ne fait qu'afficher les instantanés reçus.

Module	Responsabilité
simulation/settings.py	Constantes d'origine (scène, capteurs, cadences) et chemins
simulation/pygame_support.py	Pygame en mode headless, sprite partagé, cache de rotation
simulation/track.py	Chargement de la carte, requêtes pixel sûres (hors carte = bordure)
simulation/car.py	Physique, collision, radars, récompense, sérialisation d'état
simulation/evaluation.py	Boucle d'évaluation d'une génération, partagée web / CLI
simulation/engine.py	Thread NEAT, pause, vitesse, historique, meilleur génome
app.py	Pages, API REST, flux SSE, médias
static/js/app.js	Canvas, graphique, contrôles, réception SSE

Flux d'exécution



Chaque frame, le moteur publie un instantané (positions, angles, vitesses, radars, fitness) sous verrou. Les connexions SSE lisent la dernière version disponible : un client lent ne ralentit jamais la simulation.

4. Algorithmes de la simulation

4.1 Perception : cinq radars

Cinq rayons partent du centre de la voiture aux angles -90°, -45°, 0°, 45° et 90° relatifs à son orientation. Chaque rayon avance pixel par pixel, 300 px au maximum, jusqu'à rencontrer un pixel blanc : la bordure. L'entrée du réseau est la distance d'impact divisée par 30.

4.2 Décision : quatre actions

Sortie	Action	Effet exact
0	Gauche	angle += 10
1	Droite	angle -= 10
2	Ralentir	speed -= 2 si speed - 2 >= 12
3	Accélérer	speed += 2

L'action retenue est l'indice du maximum des quatre sorties du réseau feed-forward. La vitesse initiale est fixée à 20 à la première frame.

4.3 Collision et récompense

Les quatre coins du rectangle orienté sont testés (demi-diagonale de 30 px, angles +30°, +150°, +210°, +330°). Un pixel blanc, ou une position hors de l'image, tue la voiture. La récompense d'une frame vaut distance / 30 ; la fitness d'un génome est la somme de ces récompenses jusqu'à son crash.

4.4 Évolution NEAT

Paramètre	Valeur
Population	30 génomes
Activation	tanh
Connexions initiales	full : 5 entrées vers 4 sorties
Mutation des poids	0.8 (remplacement 0.1, puissance 0.5)
Ajout connexion / nœud	0.5 / 0.2
Seuil de compatibilité	2.0
Élitisme / survie	3 / 0.2
Stagnation	20 générations, 2 élites par espèce
Frames par génération	1 200 maximum

4.5 Boucle d'une génération

```
creer un reseau et une voiture par genome ; fitness <- 0
repeter :
    pour chaque voiture : activer le reseau, appliquer l'action
    pour chaque voiture vivante : avancer, fitness += recompense
    si aucune voiture vivante : stop
    si 1200 frames atteintes : stop
NEAT reproduit ensuite la population (population.run)
```

5. Refactoring : avant / après

Avant (main.py)	Après
1 script de ~230 lignes mêlant physique, NEAT et rendu	paquet simulation/ (6 modules) + serveur + client
boucle Pygame bloquante, une seule fenêtre	moteur dans un thread, pilotable depuis le navigateur
aucune API, aucun test	API REST + SSE, 26 tests automatisés
chemins relatifs au répertoire courant	chemins pathlib absolus, indépendants du cwd
sys.exit() pour quitter	exception dédiée et arrêt propre du thread
état global current_generation	état verrouillé, instantanés cohérents
rappel visuel dans la fenêtre uniquement	KPI, graphique, historique et téléchargement

Le découpage suit une règle simple : chaque module peut être testé seul. La physique ne connaît ni Flask ni NEAT ; le moteur ne connaît pas HTTP ; le serveur ne contient aucun calcul de simulation.

6. Corrections de bugs

Problème	Impact	Correctif
no_fitness_termination absent de config.txt	crash immédiat avec neat-python 2.0	clé ajoutée (False, comportement historique)
get_at() hors de la carte	IndexError qui tue le processus	hors carte traité comme bordure

Problème	Impact	Correctif
couleur comparée en 3 ou 4 canaux	collisions non détectées selon la surface	comparaison RGB normalisée
chemins 'car.png' et 'map.png'	démarrage dépendant du cwd	chemins absolus via settings
sys.exit(0) dans la boucle	arrêt brutal, thread non libéré	SimulationStopped + join
compteur de population écrasé	état non sérialisable	_population et _neat_population séparés
aucune sauvegarde du meilleur génome	apprentissage perdu	pickle automatique + téléchargement

La borne verticale position[1] = min(position[1], WIDTH - 120) de l'original est conservée volontairement : la corriger aurait modifié la trajectoire des voitures. Les requêtes hors carte étant désormais sûres, elle ne provoque plus de crash.

7. Améliorations et performances

- Cache des rotations de sprite : la rotation était recalculée pour chaque voiture à chaque frame ; les angles étant des multiples de 10°, un cache par angle donne exactement les mêmes pixels.
- Vitesse de simulation découplée de la physique : N frames de physique entre deux publications, sans changer l'ordre des calculs.
- Changement de carte sans redémarrage du processus.
- Logs structurés : une ligne par génération, bruit pygame et NEAT supprimé.
- Génome sauvegardé en mémoire et sur disque à chaque nouveau record.
- Mode CLI pour entraîner sans navigateur, avec la même boucle d'évaluation.

Mesure sur la machine de génération : environ 9.6 ms par frame de physique pour 30 voitures (0 encore en vie après 60 frames). À 60 frames/s, la simulation reste dans son budget, même en 4×.

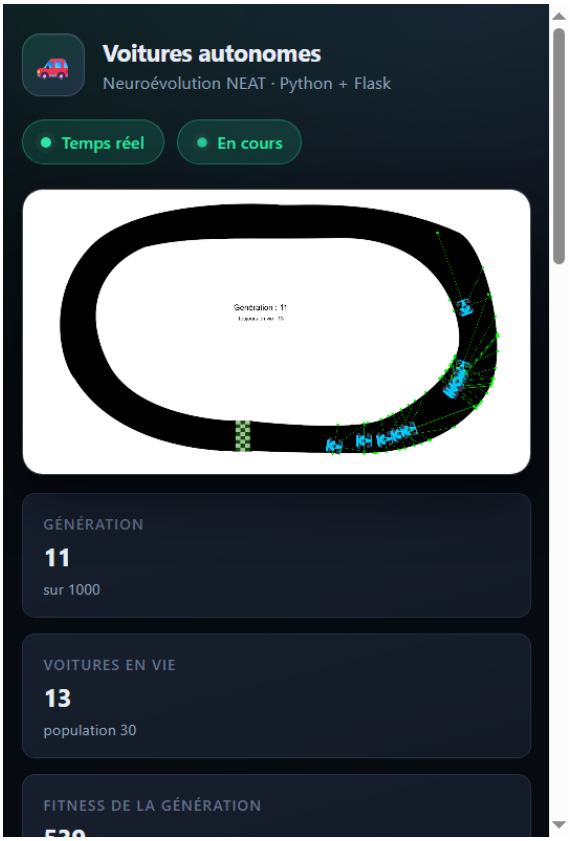
Comparaison A/B dans les mêmes conditions : 8,2 ms par frame pour le script d'origine, 9,2 ms pour la version refactorée. Le surcoût d'environ 1 ms correspond aux contrôles de bornes qui évitent les plantages hors carte — un choix assumé : la robustesse passe avant un gain marginal. Le cache de rotation de sprite réduit malgré cela les appels à pygame.transform.rotate.

8. API HTTP et flux temps réel

Méthode	Route	Rôle
GET	/	interface web
GET	/healthz	état du serveur et de la simulation
GET	/api/config	cartes, vitesses, paramètres
GET	/api/state	instantané complet
GET	/api/stream	flux SSE (une frame JSON par événement)
GET	/api/history	historique de fitness
POST	/api/start · pause · resume · reset	commandes de pilotage
POST	/api/settings	vitesse ou carte
GET	/api/best-genome	téléchargement du génome (.pkl)
GET	/media/...	carte et sprite

Le flux SSE envoie des instantanés compacts (positions arrondies, radars entiers). Un keep-alive commenté est émis toutes les 15 secondes pour traverser les proxys. Si EventSource est indisponible, le client bascule automatiquement sur un sondage de /api/state.

9. Interface et prise en main



Interface : piste, voitures et radars, indicateurs, courbe de fitness et panneau de contrôle.

Prise en main

- Démarrer / Pause / Reprendre : lance ou suspend l'évolution.
- Réinitialiser : nouvelle population, génération 0.
- Vitesse : de 0.5× à 8×, sans effet sur les calculs.
- Carte : cinq pistes, le changement relance la simulation.
- Radars : affiche ou masque les cinq capteurs.
- Télécharger le meilleur génome : fichier .pkl réutilisable.
- Raccourcis : Espace (pause), R (réinitialiser).

10. Validation par les tests

La suite compte 26 tests exécutables sans navigateur ni affichage, en environ deux secondes :

```
python -m unittest discover -s tests -t . -v
```

Suite	Ce qui est vérifié
test_parity	trajectoires rejouées depuis le main.py d'origine, comparaison au bit près
test_track	dimensions, bordure blanche, hors limites, validation des cartes

Suite	Ce qui est vérifié
test_engine	générations, limite de frames, pause, reset, génome, validations
test_api	page, configuration, commandes, erreurs 400, SSE, médias, historique

Parité avec l'implémentation d'origine

Les fixtures sont produites par tools/parity_fixtures.py, qui exécute le Car du main.py original avec une politique de pilotage déterministe, puis enregistre chaque frame. Le test rejoue les mêmes actions avec le code refactorisé et compare positions, centre, angle, vitesse, distance, coins, radars, entrées du réseau et récompense.

Fixture	Piste	Frames comparées
map_trajectory	map.png (piste réelle)	400
arena_trajectory	arène ouverte générée	66
ring_trajectory	anneau généré	400

11. Limites et perspectives

- Le serveur Flask de développement n'est pas prévu pour la production : utiliser un serveur WSGI (gunicorn, waitress) avec un worker unique, le moteur étant attaché au processus.
- Une seule simulation à la fois par processus : le moteur est un singleton applicatif.
- Pas de reprise d'un entraînement complet après redémarrage : seul le meilleur génome est conservé.
- Le flux SSE transporte du JSON : un format binaire (WebSocket) réduirait la bande passante pour de grandes populations.
- Pistes : rejeu d'une génération enregistrée, comparaison de plusieurs cartes en parallèle, export vidéo.

Annexe A. Commandes utiles

```
# installation
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# lancer l'application
python app.py
HOST=0.0.0.0 PORT=3000 python app.py

# tests
python -m unittest discover -s tests -t . -v

# entrainement sans navigateur
python train.py --generations 30 --map map3.png

# regenerer la documentation
pip install reportlab python-pptx pillow
python tools/make_docs.py
```

Annexe B. Structure du projet

```
app.py                  serveur Flask (pages, API, SSE)
train.py                entrainement NEAT headless
config.txt              configuration NEAT
requirements.txt
simulation/
  settings.py           constantes et chemins
  pygame_support.py     Pygame headless, sprite, cache de rotation
  track.py              carte et requetes pixel sures
  car.py                physique, capteurs, recompense
  evaluation.py         boucle d'evaluation d'une generation
  engine.py             thread NEAT, pause, vitesse, instantanes
templates/index.html    interface
static/css/style.css
static/js/app.js        canvas, SSE, graphique, controles
assets/                 car.png + 5 cartes
tests/                  26 tests + fixtures de parite
tools/                  make_docs.py, parity_fixtures.py
docs/                   documentation.pdf, presentation.pptx
runs/                   meilleur genome + logs
```